

OSQL Utility

By Don Schlichting

This article will explain and define the OSQL utilities ability to run scripts for administration and batch work when using Microsoft SQL Server.

Introduction

If you have repetitive tasks to run, multiple SQL servers to administer, or a very large sequence of commands to execute, then the OSQL utility may be the tool to use. OSQL is capable of running both scripts and interactive commands. It is started from the command line and can be executed manually or by a scheduled task. With over twenty-five switch commands, OSQL can usually be configured to execute, as your application requires. OSQL does not have a user interface. So many times, scripts are created in Query Analyzer, saved and then run by OSQL.

OSQL vs. ISQL and Query Analyzer

There is a great deal of overlap between ISQL and OSQL. Both support input scripts, output scripts, and most of the same switch arguments. OSQL has no interface. It will only accept a typed command line, or a saved script. In spite of this disadvantage, sometimes ISQL and Query Analyzer cannot accomplish the required task. Working with MSDE is one example. Query Analyzer is not included with the Microsoft Desktop Engine. When developing an application on MSDE, or needing to do MSDE administration, the OSQL utility is the only tool included. Another key difference between ISQL and OSQL is the base library each tool was built on. ISQL is developed on the DB Library, as opposed to OSQL being developed on ODBC. The DB Library works at the SQL 6.5 standard. This difference means ISQL, or any application developed on the DB Library, does not support some of the new SQL 2000 features. The entire list of unsupported features can found in Books on Line under the title "Connecting Early Version Clients to SQL Server 2000." Some of the main limitations of ISQL include char and varchar defined greater than 255 bytes will be non accessible, big ints will be converted to decimals, sql_variants will be converted to nvarchar, XML results may not be retrieved, and bit fields that are null will be reported as not null with a value of 0. OSQL and Query Analyzer will support all of the SQL 2000 features.

OSQL ad hoc query examples

In these beginning tests, we will execute ad hoc queries from the command line. This first command assumes SQL server is local on your machine and you have Windows trusted connection, rather than a SQL uid and password. Open the command prompt and enter:

```
OSQL -E -Q "SELECT * FROM syslogins"
```

The contents of the syslogins table should scroll down the command window. In the above statement, -E tells OSQL to used a trusted connection rather than a SQL uid. The -Q is the query statement. Because no database was specified, master was used. To specify a database, change the statement to:

```
OSQL -E -d pubs -Q "SELECT * FROM authors"
```

The switches are case sensitive. Moreover, many times a lower case letter has no relation to an upper case letter. Lower case p is used for print performance statistics while an upper case P is used to specify a SQL password.

To use SQL security rather than Windows security, remove the -E and change the statement to:

```
OSQL -U sa -P secret -d pubs -Q "SELECT * FROM authors"
```

OSQL scripts

In these next examples, we will create and save TSQL scripts, and then run them from OSQL. Query Analyzer is a standard choice for script creation because of the color coding. Open Query Analyzer and enter:

```
USE pubs
GO
SELECT * FROM authors
GO
```

Save this script to your hard drive, and then from the command line, enter an OSQL statement using the -i switch to specify an input file. The authors table should be returned.

```
OSQL -E -i c:\temp\q1.sql
```

```
C:\>osql -E -i c:\temp\q1.sql
1> 2> 1> 2>  au_id      au_lname
            address
-----
172-32-1176 White
            10932 Bigge Rd.
213-46-8915 Green
            309 63rd St. #411
238-95-7766 Carson
            589 Darwin Ln.
267-41-2394 O'Leary
            22 Cleveland Av. #14
274-80-9391 Straight
            5420 College Av.
341-22-1782 Smith
            10 Mississippi Dr.
409-56-7008 Bennet
            6223 Bateman St.
427-17-2319 Dull
            3440 Blaine St.
```

The results of the query can be captured to an output file, rather than appearing on the screen. Change the command line to include the -o parameter, for output.

```
OSQL -E -i c:\temp\q1.sql -o c:\temp\results.txt
```

OSQL should create a text output file. The -u switch can be used to control the output file being either Unicode or OEM.

System Commands

Operating system commands can also be executed from inside the TSQL script. The key !! is used to specify this. Change the Query Analyzer script and save it as:

```
!! dir c:\temp
GO
USE pubs
GO
SELECT * FROM authors
GO
```

Now our output file will include the directory listing of the temp folder in addition to the authors' results. Also, note this script will not run in Query Analyzer. The !! directive is not supported. Query Analyzer color coding is helpful in code layout, but the script testing will need to be done from OSQL.

Error Handling

OSQL supports the RAISERROR command for returning custom error messages. To use raise error, the database name, id, error severity and state should be included. Using RAISERROR will cause the script to terminate. Modify the Query Analyzer script to:

```
!! dir c:\temp
GO
DECLARE @DBID int
SET @DBID = DB_ID()
DECLARE @DBNAME nvarchar(128)
SET @DBNAME = DB_NAME()
RAISERROR('my error', 18, 127, @DBID, @DBNAME)
USE pubs
GO
SELECT * FROM authors
GO
```

Running the script will now output our directory listing, from the "!! dir c:\temp" command, followed by the raise error. The remaining script, changing to pubs and selecting from authors will not occur. RAISERROR will terminate the script.

Leaving a script can also be done by calling QUIT or EXIT from inside OSQL. Neither of these will return an error code, but EXIT can execute a statement prior to quitting. For example: EXIT(SELECT @@ROWCOUNT).

Conclusion

When administering the Microsoft Desktop Engine, OSQL is a free way to run statements. For standard SQL Server environments, OSQL can be used to help automate long or repetitive tasks by reusing scripts. OSQL is also a good choice to run database setup scripts during application install procedures.